Optimal Freelancing

AlgoExpert · Easy · Greedy Algorithms

The problem

- Each job takes one day and has a deadline (days left) and a payment.
- Working at most 7 days, one job per day, maximise profit.
- Greedy: take jobs by payment descending and place each in the latest free day on or before its deadline.

Intuition

- Greedy by payment: take the most valuable jobs first and schedule each as late as its deadline allows.

Approach

- 1. Sort jobs by payment, highest first
- 2. Keep seven day slots
- 3. Place each job in the latest free day before its deadline
- 4. Sum the payments of placed jobs

Java solution

```java
public class OptimalFreelancing {
    public static int optimalFreelancing(List<Map<String, Integer>> jobs) {
        final int days = 7;
        boolean[] taken = new boolean[days];
        int profit = 0;
        List<Map<String, Integer>> sorted = new java.util.ArrayList<>(jobs);
        sorted.sort((a, b) -> Integer.compare(b.get("payment"), a.get("payment")));
        for (Map<String, Integer> job : sorted) {
            for (int d = Math.min(job.get("deadline"), days) - 1; d >= 0; d--) {
                if (!taken[d]) { taken[d] = true; profit += job.get("payment"); break; }
            }
        }
        return profit;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(1) (7 slots)

Recap

- Greedy by payment: take the most valuable jobs first and schedule each as late as its deadline allows.
- Complexity: O(n log n)